Одно приложение

Образ, контейнер и HTTP

Занятие 17.10.2026. Пишу от своего лица; код набирал вручную через Vim и разбирал с преподавателем. Показанные сборка, запуск, ответ и логи отделены от ещё не проверенного самостоятельного применения.

1. Понятная задача вместо нескольких сервисов

Я попросил вернуться от Compose к Docker: не хватало целостной картины одного приложения и назначения действий. Для примера выбрали lab-info - маленький HTTP-сервис, возвращающий своё имя и версию. Такой ответ позволяет проверять, какой выпуск приложения запущен.

Код лежит в ~/docker-one-app на VM linux01. Планировалось упаковать его вместе с Python, запустить одним контейнером и проверить ответ. Python для приложения берётся из образа; отдельно устанавливать его на Ubuntu для этого примера не требовалось.

```
app.py + Dockerfile -> сборка образа lab-info:v1
                    -> запуск контейнера lab-info
                    -> процесс python app.py
                    -> HTTP-запрос и запись в журнале
```

2. Что делает Dockerfile

Разобранный учебный вариант:

```
FROM python:3.14-alpine
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

- FROM выбирает базовый образ с Python; вариант python:3.14-alpine поддерживает ARM64.
- WORKDIR задаёт /app рабочим каталогом для последующих инструкций и запуска.
- COPY добавляет файл app.py из каталога сборки в рабочий каталог образа.
- CMD задаёт команду по умолчанию при запуске контейнера. Она не запускает постоянный сервер во время сборки образа.

Полный Dockerfile отдельным выводом не показан. На скриншоте сборки видны COPY и CMD; HTTP-заголовок позднее подтверждает Python 3.14.8. Сам заголовок не доказывает вариант базового дистрибутива Alpine.

3. Сборка и запуск - разные действия

```
sudo docker build -t lab-info:v1 .
sudo docker run -d --name lab-info \
  -p 127.0.0.1:8145:8000 lab-info:v1
```

build создаёт образ; -t назначает имя/тег, точка выбирает текущий каталог сборки. run создаёт и запускает контейнер из образа. -d оставляет его в фоне, --name назначает имя. Публикация направляет порт 8145 VM на порт 8000 приложения внутри контейнера.

Образ lab-info:v1 - подготовленный пакет. Контейнер lab-info - запущенный экземпляр с процессом и собственным состоянием. Их имена похожи, но это разные объекты Docker.

Что происходит внутри приложения

4. Код, который я набрал

На показанном app.py тело содержит перевод строки перед закрывающей фигурной скобкой JSON; это допустимый пробельный символ:

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class App(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b'{"service":"lab-info","version":"1.0"\n}'
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

HTTPServer(("0.0.0.0", 8000), App).serve_forever()
```

5. Новые конструкции Python

- import подключает имена из стандартной библиотеки: HTTPServer принимает соединения, BaseHTTPRequestHandler даёт основу обработки HTTP.
- class App описывает наш класс обработчика. Базовый класс в скобках даёт готовые возможности; это наследование.
- def определяет метод. Библиотека вызывает do_GET при запросе GET. self - конкретный объект обработчика, работающего с клиентом.
- Отступы задают вложенность. body - переменная, знак = присваивает значение.
- b перед кавычками создаёт bytes: готовые байты с текстом JSON, а не Python-словарь. Перевод строки задаётся символами обратной косой черты и n.

6. Откуда берётся HTTP-ответ

send_response готовит статус 200. Content-Type сообщает клиенту формат тела JSON, Content-Length - размер в байтах. В str(len(body)) сначала считается длина body, затем число превращается в текст заголовка. end_headers завершает и отправляет заголовки; wfile.write отправляет тело клиенту через сетевой поток, не записывая файл на диск.

Последняя строка создаёт HTTPServer с адресом/портом и обработчиком App. Адрес 0.0.0.0 означает слушать все IPv4-интерфейсы текущей среды, здесь контейнера. serve_forever запускает ожидание и обработку запросов. Без всей последней строки определения класса сами сервер не запустят.

```
Python запускает файл -> создаёт сервер -> ждёт запрос
GET -> do_GET -> статус/заголовки/тело -> ответ -> ожидание дальше
```

Связь с Docker стала конкретной: CMD запускает Python, а работающий серверный цикл удерживает процесс приложения. Поведение при остановке и обновлении этого приложения ещё предстоит проверить.

Что я показал

7. Сохранение файлов и рабочий инструмент

Я предпочитаю Vim и набираю код вручную. Сначала Vim передал wq оболочке как внешнюю команду; получен command not found и exit 127. Наставник объяснил различие :!wq и :wq. Затем E45 сообщил о readonly; после подсказки :wq! я сообщил об успешном сохранении, а скриншот 18:48 с cat app.py подтвердил файл на диске. Причина включённого readonly отдельно не установлена.

Я попросил разбирать также код, поскольку не программист и хочу конспектировать конструкции для дальнейшего применения. Правило добавлено в FORMAT. Отдельный конспект app.py подготовлен; изучение кода остаётся частью практики Docker.

8. Сборка и контейнер

На скриншоте 19:00 видны Successfully built 0ee63dec8d25 и Successfully tagged lab-info:v1. Перед запуском ss -ltnp с фильтром порта 8145 показал только заголовок, без слушающего сокета.

Run вернул ID, начинающийся с 348461c329b7. В docker ps -a контейнер lab-info использовал образ lab-info:v1, команду python app.py, имел Up 9 seconds и публикацию 127.0.0.1:8145 -> 8000/tcp.

9. Ответ приложения и журнал

```
curl -i --max-time 3 http://127.0.0.1:8145/
sudo docker logs lab-info
```

Показаны HTTP/1.0 200 OK, Server: BaseHTTP/0.6 Python/3.14.8, Content-Type: application/json, Content-Length: 39. Тело содержит service lab-info и version 1.0 с переводом строки перед последней скобкой.

В журнале показан клиент 172.17.0.1, время 17/Oct/2026 16:59:50 и запрос GET / HTTP/1.1 с кодом 200. Это доказывает обработку конкретного запроса приложением. Код клиента HTTP/1.1 и версия ответа HTTP/1.0 в этом примере не обязаны совпадать.

10. Итог и продолжение

Изучено и отработано: сохранение кода, сборка своего образа, запуск одного приложения, публикация порта, HTTP-ответ и журнал. Код и действия разбирались с преподавателем; новую самостоятельную настройку или диагностику этот результат не доказывает, уровни не повышаются.

Следующий шаг остаётся на этом приложении: объяснить и проверить связь исходника, образа и контейнера при изменении версии, затем жизненный цикл и данные. Удаление старых контейнеров и настройка Compose сюда не входят. Изменения VM выполнял я; преподаватель сервером не управлял.

Источники: Образ Docker, Dockerfile, официальный образ Python, HTTP-сервер Python, классы, bytes, цикл сервера.